

Surrogate-assisted Reinforcement Learning for Radiation Shielding Design

Alexander Whitesides¹

¹Georgia Institute of Technology, Atlanta, GA,
awhitesides3@gatech.edu

INTRODUCTION

The overarching goal of this research is to aid in the deployment of Floating Nuclear Power Plants (FNPPs) by seeking to optimize their radiation shielding design. The economic feasibility of FNPPs is a main barrier to their deployment. Designing a shield that optimizes for cost, while adhering to dose limits, is a route that contributes to alleviating this barrier. This research seeks to employ a rudimentary machine learning (ML) methodology for application towards nuclear technical design processes.

The narrowed focus for this paper is on exploring the coupling of a Monte Carlo (MC) environment with a reinforcement learning (RL) algorithm to perform optimization on nuclear reactor designs. This will be explored through the case study of optimizing the shield design for a marine SMR reactor, the Nuclear Ship Savannah (NSS) [1].

METHODOLOGY

RL is a type of ML where an agent learns an environment by performing an action which results in a particular state and reward. Fig. 1. illustrates the RL process which is backed mathematically by the Markov Decision Process (MDP). The agent's goal is to maximize the reward which acts as a fitness function for the environment which the agent is exploring.

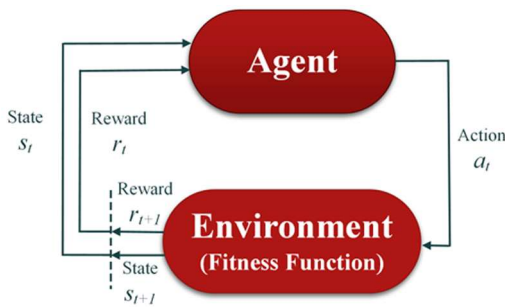


Fig. 1. RL flow for optimization (Reproduced from [2]).

The main benefit of RL is the development of an agent which has dynamic learning capabilities. This is impactful in the nuclear design space where expensive simulation codes are a necessity. With the rise in computational power and ML exploration, RL is now a practical optimization methodology to reduce computation time. Current optimization methodology comprises of leveraging evolutionary algorithms (EA) or simply following a manual “trial-by-

error” approach. The ability to “grow” an agent simultaneously with a technical design is unique of RL and presents a long-term approach to design methodology. If a nuclear plant operates for 60 years, why not have an optimization agent that has learned the plant over that 60-year lifetime?

The specific RL algorithm that is explored in this research is the proximal policy optimization (PPO) algorithm [3]. The details on this algorithm and python compatibility are provided at the NEORL website [2]. PPO was chosen for its ability to handle both discrete and continuous spaces. Additionally, its “reinforcement learning flow for optimization” follows the same structure as the MDP. PPO originated because the previous methods for RL optimization were not scalable for large, continuous action spaces, such as nuclear shielding problems. The Q-learning approach can’t handle continuous action spaces like the thickness of a shield layer or dose results [3]. The trust region policy optimization (TRPO) is suitable for continuous action spaces, but it uses second-order calculations to predict how a proposed action will affect the long-term optimization to inform the next action [3]. This is not efficient for optimization. The PPO algorithm utilizes a clipping method that limits how much a single action can direct the optimization process. This allows it to use only first-order equations allowing for scalable optimization for large problems. PPO is a happy medium between the Q-learning and TRPO algorithms of the past [3].

The key input of the PPO algorithm is the environment which contains the fitness function. Additional inputs include hyperparameters for the RL. The default values for RL hyperparameters are used as a sensitivity analysis of these hyperparameters is not in scope of this work. The four key functions of PPO are the ‘learn’, ‘predict’, ‘save’, and ‘load’. The ‘learn’ function is where the MDP takes place and is used to train the model on the provided environment. The only variable hyperparameter that this research considers is the ‘total timesteps’ which is the key cost driver of PPO and essentially describes the total number of actions applied to the environment. The ‘predict’ function calculates a reward from a given state and is used to evaluate the model. The ‘save’ and ‘load’ process the agent state into a retrievable data file for future implementation (unique advantage of RL).

This research leverages the tool OpenNeoMC which couples the radiation transport code (OpenMC) and the optimization algorithm of PPO in a python implementation [4]. Initially, the OpenMC model was imbedded in the PPO environment (fitness function), but this led to inefficiencies

in the PPO learning process. Instead, a surrogate approach was implemented. An OpenMC sample solution space of 10-200 runs, refined by a Gaussian Process Regressor (GP), is embedded into the PPO environment. Then the PPO agent is initialized, and the RL process begins. Fig. 2 outlines the workflow for the RL code.

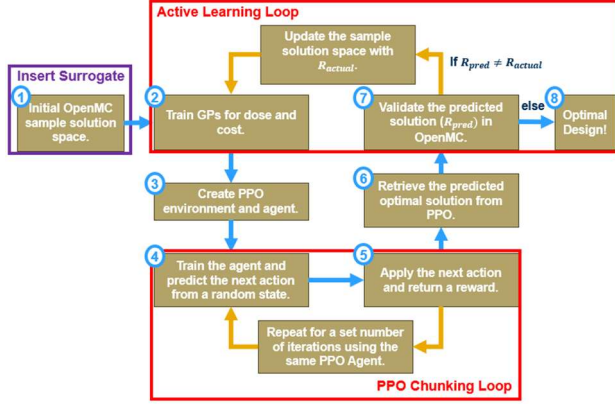


Fig. 2. RL code process flow.

Steps 1-8 provide a linear workflow for developing an optimal design from OpenMC runs and a PPO RL implementation. The ‘PPO Chunking Loop’ serves as a method to evaluate the performance of the PPO agent at designated checkpoints through the entirety of the agent learning process in step 4. Once the agent has ‘fully’ learned the model, a predicted solution is obtained in step 6. The state of this predicted solution is validated in OpenMC (step 7). If the validation process is successful (accuracy metric is met) then the process is complete. If the validation is unsuccessful then the ‘Active Learning Loop’ is enforced. This loop appends the state from the failed predicted solution, along with its actual solution to the OpenMC sample solution space. Then the GP is retrained on the updated solution space in step 2. The PPO process is repeated until a successful validation (steps 3-7).

As previously addressed, the key metric of the agent’s performance in RL is the reward. For the shielding optimization code, the goal is to achieve the largest reward or in this case as seen in Eq. 1, the smallest negative reward.

$$Reward = -cost - Penalty. \quad (1)$$

The goal is to minimize the cost of the shield while adhering to a dose limit. A failure to meet the dose limit results in a penalty. The penalty philosophy is a crucial component to the performance of the PPO agent. The penalty policy was iterated on throughout the development of this RL code. Ultimately, a continuous policy rather than a binary policy was considered as it is more forgiving during the RL process. Equation 2 provides the penalty policy where the dose constraint, $dose_c$, is compared to the dose result from each OpenMC run, $dose_i$.

$$Penalty = \max\left(0, A * \frac{dose_i - dose_c}{dose_c}\right). \quad (2)$$

This penalty policy treats dose as a soft constraint. However, its harshness can be adjusted by the magnitude of the scaling coefficient, A . The penalty policy was tested with the coefficient values of 10, 100, and 1000. This scales the effect of the penalty if the $dose_c$ is not met. Ultimately, an A of 1000 was used in the final analysis.

This performance of this code will be explored via the case study of the Nuclear Ship Savannah (NSS) – a commercial nuclear propulsion marine vessel with a 70MW PWR that was in active service 1962-1972. The PPO agent’s goal will be to optimize the design under the reward criteria in Eq. 1. Several papers have explored this case study for shielding optimization, but with the use of EAs. Although they use EAs, References 5 and 6 have inspired the physical problem setup. The physical problem involves a 1-D model depicted in Fig. 3.

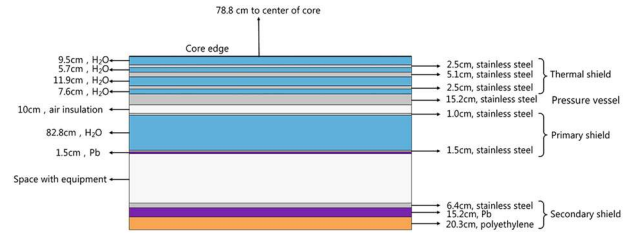


Fig. 3. Depicts the shield layers of the NSS shield design (Reproduced from [5]).

The design can be segmented into three sections based off proximity to the core: thermal shield, primary shield, and secondary shield. For this paper, only the thermal shield section is considered, limiting the MC analysis to strictly neutron fast flux analysis. The exact parameter space is organized in Table I. This is the same design specifications listed in reference 6, except for reduced parameter ranges. This is necessary to streamline the PPO process.

Table I. NSS Shield Design Specifications

Material	Initial Value [cm]	Optimal Parameter	Range [cm]
Core	78.74	n/a	n/a
Water 1	9.525	x_0	0-10
SS 1	2.54	x_1	0-10
Water 2	5.715	x_2	0-10
SS 2	5.08	x_3	0-10
Water 3	11.938	x_4	0-10
SS 3	2.54	x_5	0-10
Water 4	7.62	x_6	0-10

The order of the shield layers is held constant meaning only the thickness of each shield layer is optimized (denoted by the ‘optimal parameter’ column).

The OpenMC geometry for the thermal shield is presented in Fig. 4.

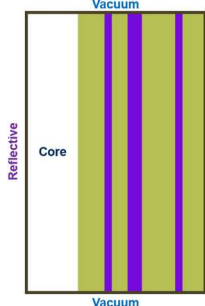


Fig. 4. 1-D Slab Geometry from OpenMC in the XZ Plane.

Assumptions of the model include an isotropic angular distribution, a constant spatial distribution along the z-axis, and a constant time distribution for the reactor core.

To determine the dose limit at each layer a ‘full core’ model was constructed with fast neutron flux tallies. This ‘full core’ model contained a fixed source and the proper core materials using the geometry from Fig. 4. The core material is simplified as a homogenous core of 4.4wt% enriched UO_2 fuel. The volume fractions are included in Reference 5. It was run once, producing the fast neutron flux (per source neutron) results in Table II.

Table II. Neutron Fast Flux Results from the Thermal Shield

Tally Location	Mean Value [-]	Absolute Error	Relative Error
Layer 1 (x_0)	$1.384 * 10^{-1}$	$2.617 * 10^{-4}$	0.0019
Layer 2 (x_1)	$9.361 * 10^{-2}$	$2.985 * 10^{-4}$	0.0032
Layer 3 (x_2)	$3.336 * 10^{-2}$	$1.803 * 10^{-4}$	0.0054
Layer 4 (x_3)	$1.745 * 10^{-2}$	$9.183 * 10^{-5}$	0.0053
Layer 5 (x_4)	$1.794 * 10^{-3}$	$4.126 * 10^{-5}$	0.023
Layer 6 (x_5)	$1.218 * 10^{-3}$	$3.593 * 10^{-5}$	0.029
Layer 7 (x_6)	$3.430 * 10^{-4}$	$2.261 * 10^{-5}$	0.066

A limitation of this paper is that the dose constraint is unitless. The source normalization is out of scope because this paper is focused on methodology rather than numerical results. Therefore, the mean value tally results from Table II are used directly in the RL code as the dose constraint, $dose_c$.

To save computational time, a surface source (file-based source) is used for the PPO runs. The surface source leverages a voided core, ignoring the radiation physics occurring inside the core, which reduces computational time. See OpenMC User’s Guide 7.3.1.1 for more details [7]. Again, this is a valid assumption for the demonstration analysis performed in this report as result trends are more important than the actual result values.

As for cost, the shield materials inform the total cost by their thickness and the cost per thickness; tabulated in Table III. To align with the scope of this paper, the cost values are arbitrary (unitless), but inspired from real-world context.

Table III. Material Costs

Material	Cost [cm^{-1}]
SS-316L (SS)	3.70
Water	0.0093

Since the model is 1-D, these cost values are multiplied by the shield layer thicknesses to yield the total cost of the shield design.

RESULTS

Initial runs considered only the first two layers in Table I. The ‘total_timesteps’ was set to 1000 and performed over 100 iterations. The thicknesses of each layer and the corresponding reward are provided in Fig. 5.

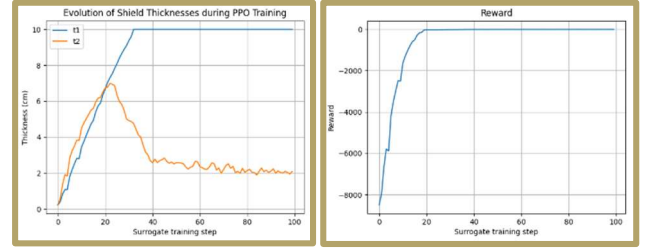


Fig. 5. 2-Layer results: thickness (left) and reward (right).

The converged reward yielded optimal thicknesses of $\sim 10cm$ and $\sim 2cm$ for layers 1 and 2, respectively. When looking at the dose and cost results in Fig. 6, it is clear what led to the converged thicknesses.

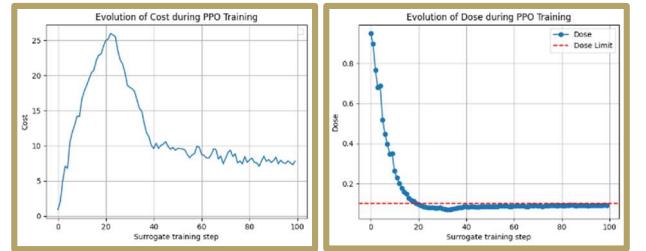


Fig. 6. 2-Layer results: cost (left), dose (right).

Around the 20th training step, the thickness of layer 2 trends down and so does the cost. In fact, the cost trend is nearly an exact replica of the layer 2 thickness trend. This is because the cost of stainless steel is nearly 3 orders of magnitude more expensive than water. This reinforces the agent to minimize the SS layer. Since water is extremely cheap and effective at slowing down neutrons, the agent seeks to maximize its contribution to the shielding design. Looking at the dose trend, the agent progresses towards the limit. Once it reaches the limit, it slightly oscillates, finding the maximum dose under the limit.

For a deeper understanding, it is important to analyze the GP’s construction of the environment which influences the

PPO agent's ability to discover an accurate optimal solution. Fig. 7 provides the mean and uncertainty of the dose map for the 2-layer case.

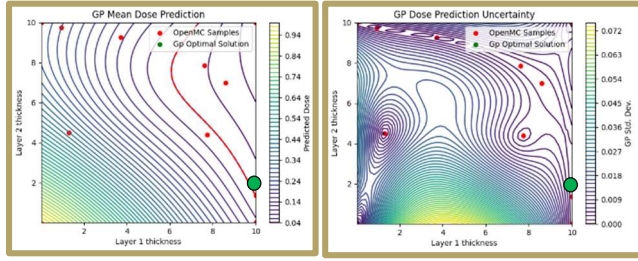


Fig. 7. 2-Layer dose map: mean (left), uncertainty (right).

The red dots are the OpenMC sample solutions provided by the surrogate. The GP fits a continuous solution map with dose ‘gradients’ covering the entire solution space (0-10cm). The green dot (enlarged for visual effect) is the optimal solution proposed by PPO. High GP uncertainty will cause the validation step to fail (step 7 of Fig. 2) and initiate the ‘Active Learning Loop’. When this occurs, it is because the GP acts as a ‘faulty’ guide for the PPO agent, leading down a false optima valley. This is the limiting factor that influences the active learning loop as the cost uncertainty is extremely low due to its simple linear relationship.

Further trials added a 3rd and 4th shield layer (Fig. 8). Largely the results are similar.

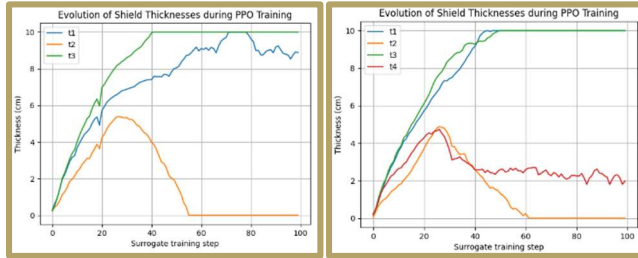


Fig. 8. 3-Layer thickness results (left), 4-Layer thickness results (right).

Hiccups do arise with increasing layers as each layer added increases the solution space by an order of magnitude. To compensate, strategic OpenMC samples are provided to the surrogate to eliminate extrapolation; strictly forcing the GP to interpolate. In the 4-layer case, 200 OpenMC samples were required as opposed to the 10 used in the other trials.

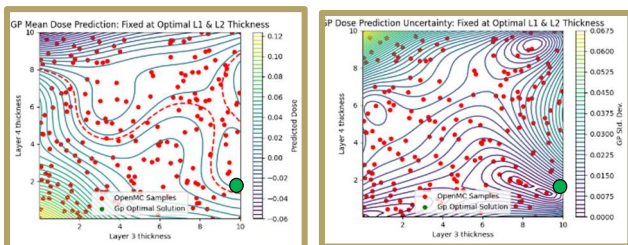


Fig. 9. 4-Layer dose map: mean (left), uncertainty (right).

CONCLUSION

Through the iterative process of learning the PPO's behavior, the performance of the agent hinges on the reward policy and the assumption of a ‘correct’ model. A ‘correct’ model implies proper variance reduction and physically realistic designs. If high uncertainty is present in the OpenMC solutions, then the physical solution provided by the optimization will not be practical.

On the other hand, a proper understanding of the expected dose limits outside each shield layer is crucial for accurately informing the penalty policy. If the scaling of the penalty is off or the $dose_c$ is not realistic, the agent may struggle to effectively learn the solution space.

Ultimately, there are many ways to optimize through the coupling of MC and RL. The end goal of this research is to provide a tool that handles the iterative nature of design changes in engineering process. Specifically, for nuclear, this tool could be used for design optimization of shielding pesky streaming paths. It likely will not be applied to initial design scoping, but rather used as a fine tuner towards the end design.

Future work will focus on exploring more layers and alternative dose constraints, including thermal flux and secondary radiation effects.

REFERENCES

1. Blizard, E. P., et al. "THE RADIATION LEAKAGE SURVEY OF THE SHIELD OF THE NUCLEAR SHIP SAVANNAH.", Aug. 1962. <https://doi.org/10.2172/4777778>
2. M. I. RADAIDEH, K. DU, P. SEURIN, D. SEYLER, X. GU, H. WANG, AND K. SHIRVAN, “NEORL: NeuroEvolution Optimization with Reinforcement Learning,” arXiv:2112.07057 (2021).
3. J. SCHULUM, F. WOLSKI, P. DHARIWAL, A. RADFORD, AND O. KLIMOV, “Proximal Policy Optimization Algorithms,” arXiv:1707.06347, Cornell Univ., Ithaca, NY (2017).
4. X. GU, M. I. RADAIDEH, AND J. LIANG, “OpenNeoMC: A Framework for Design Optimization in Particle Transport Simulations Based on OpenMC and NEORL,” Ann. Nucl. Energy, 180, 109450 (2023).
5. X. WU, Y. YANG, S. HAN, Z. ZHAO, P. FANG, AND Q. GAO, “Multi-Objective Optimization Method for Nuclear Reactor Radiation Shielding Design Based on PSO Algorithm,” Ann. Nucl. Energy, 160, 108404 (2021).
6. Y. WU, B. LIU, X. SU, S. TANG, M. YAN, AND L. PAN, “Study on Multi-Objective Optimization Method for Radiation Shield Design of Nuclear Reactors,” Nucl. Eng. Technol., 56(2), 520–528 (2024).
7. P. K. ROMANO, N. E. HORELIK, B. R. HERMAN, A. G. NELSON, B. FORGET, AND K. SMITH, “OpenMC: A State-of-the-Art Monte Carlo Code for Research and Development,” Ann. Nucl. Energy, 82, 90–97 (2015).